AI201 PA 3 Report
Implementing the Backpropagation Algorithm

Deomary Angelo Franco (2023-22049)

Date of Submission: April 02, 2025

1. INTRODUCTION

Artificial Intelligence, also known as AI, has been a major trend and talk of the town in the twenty-first century. It encompasses a wide range of technologies and methodologies aimed at developing systems capable of performing tasks that typically require human intelligence. But how does AI perform these tasks with full autonomy? The answer lies in its learning capabilities. While AI can only perform tasks on how it is trained, if performed properly, these systems are able to generalize well when facing new but similar problems.

If their capabilities are dependent on how it learns, then how do they learn? Let use look at a simple intelligent system such as a dog and cat identifier. If we have an image of either a dog or a cat, a system is said to generalize well if it is able to identify if the animal in the photo regardless of its orientation, color, size, breed, or even what the background image is. To do this, the system must first be trained on numerous labeled examples–all of which must be unique from other. Similar to organic human beings, they encounter different types of dogs and cats throught their lifetime such that expand their definition of what those pet animals should look like. Likewise with intelligent systems, we must feed different images of cats and dogs and let the system gradually understand what truly defines these animals.

While biological creatures have a complex way of learning and are yet to be fully understood, AI researchers have developed multiple algorithms for AI systems to learn. One learning algorithm that is popularly used is the backpropagation algorithm. To understand backpropagation, let us first introduce the multilayer perceptron (MLP), a kind of artificial neural network. In general, artificial neural networks are a series of layers where each layer is composed of multiple neurons or nodes–carriers of data. Each layer is connected to other layers via the neurons. Inference happens by passing these data throughout the different connected nodes. The MLP, which is colloquially called the "vanilla" artificial neural network, is part of the family of neural networks, where each layer is connected via the adjacent layers. See in Figure 1 a simple multilayer perceptron with 2 hidden layers and its anatomy.

Shown in Figure 1 is a simple anatomy of a MLP that will be discussed later.

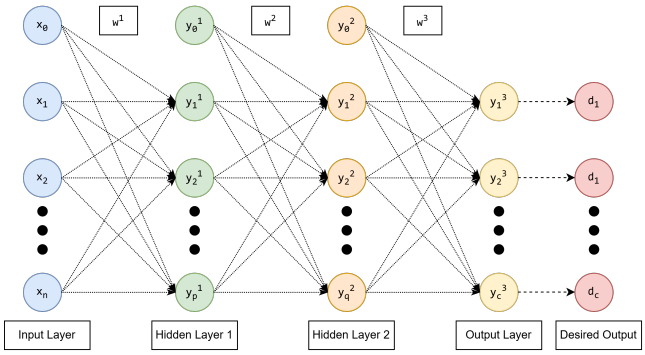

Figure 1: Schematic of the MLP–"vanilla" neural network. Shown is a MLP with two hidden layers. There are four main layers, the input layer, hidden layer 1, hidden layer 2, and output layer.

2. OBJECTIVES

The objective of this paper is to discuss the implementation of a Multilayer Perceptron, together with the forward propagation and back propagation algorithms. Using the implemented MLP, generate two networks where both layers have two hidden layers. Network A uses `tanh-tanh-sigmoid` while Network B uses `leaky_relu-leaky_relu-sigmoid` for its hidden layers and output layer. Additionally, the following is aimed to be achieved:

- Explain the partitioning of the dataset and handling of the imbalanced dataset.
- Hyperparameter optimization or how the hyperparameters hidden layer nodes, learning rate η, and momentum constant α were determined.
- Show and explain the learning curves for the trained models.
- Compare training duration of Network A and Network B.

- Show the difference of Network A and Network B via different metrics such as Confusion Matrix, Accuracy, Precision, Recall, F1 scores, Matthews Correlation Coefficient.

- Discuss conclusion.

3. METHODOLOGY

Dataset given is first processed by oversampling the minor class so that it can be used effectively. Once balanced, it is split into validation and training where model is trained on the training set but metrics are evaluated at the validation. While training is performed to have a good model, hyperparameter optimization is used to improve the model more. The validation dataset is then used to compute for the other metrics to give us insights on the performance of the improved model.

3.1 Data Preprocessing

The dataset used for this study has 3486 instances where each instance has 354 features. The class labels of the dataset $y \in \{1, 2, \ldots, 8\}$ are available for each instance. The dataset however is highly imbalanced as seen from its distributions displayed by Table 1. Utilizing these data is not preferable. Advanced techniques are required to solve this issue.

Label	Count	Prop. (%)
1	1625	46.62
2	233	6.68
3	30	0.86
4	483	13.86
5	287	8.23
6	310	8.89
7	52	1.49
8	466	13.37
SUM	3486	100.00

Table 1: It can be observed here that Class 1 dominates the dataset–comprising of almost half the data. Class 3 and Class 7 on the other hand are considered negligible. Using these dataset for the multilayer perceptron training would yield bad results as the model would lean towards the dominant class–class 1.

One might think that replicating/duplication the minority oversampling the minority with replacement would fix this problem, however, it does not actually help the model in generalization since no new information is added during training. To solve this undesirable imbalance, a technique called "Synthetic Minority Oversampling Technique", or simply SMOTE, is employed. This technique works by drawing an imaginary line between a sample and one of its nearest neighbors and then generating a synthetic data point at a random point along the line. This method works in practice since you diversify your minor class but still within the limits of the original dataset.

To have a balanced dataset using SMOTE, we use the counts of the majority class and use that as benchmark count for the remaining class. Since the major class is Class 1 with count of 1625, then we should have a total of 13,000 instances with each class having 1625 instances.

Next, we split the dataset into randomly into training and validation dataset. This random split will also be applied to the labels, and it is crucial to ensure that the labels still match the dataset. This is satisfied by using their indices instead. We shuffle the the indices, split the indices into training and validation, and then use those split indices to determine which dataset goes into the training, and which goes into the validation. The selected count of validation is 800, with the remaining 12,200 as the the training. Since this shuffling is random, it is expected to have almost equal distributions for each class. See Table 2 for the final counts and proportions.

	Training		Validation	
Label	Count	Prop. (%)	Count	Prop. (%)
1	1504	12.33	121	15.13
2	1528	12.52	97	12.13
3	1539	12.61	86	10.75
4	1513	12.40	112	14.00
5	1516	12.43	109	13.63
6	1536	12.59	89	11.13
7	1530	12.54	95	11.88
8	1534	12.57	91	11.38
SUM	12200	100.00	800	100.00

Table 2: After training the distribution of each class is now approximate balanced. This is the desired distribution to train the model.

3.2 Implementation

The main algorithms utilized by a Multilayer Perceptron is the forward propagation and back propagation. Forward propagation is used to predict the output based from the input. Data travels from one layer to another by performing a series of weighted sum plus the bias and then "squeezed" to a certain range by some activation function. Popular choices for the activation function is presented in Table 3. The output then becomes the input for the next layer. The sequence continues up until the output layer is reached. See Figure 2 and Equation 1. The forward propagation shown here is in vector form to to allow parallel calculations.

$$y = XW \tag{1}$$

Activation Functions	$\phi(x)$	$\phi'(x)$
sigmoid/logistic	$\frac{1}{1+e^{-ax}}$	$a\phi(x)(1-\phi(x))$
tanh	$a\tanh(bx)$	$\frac{b}{a}(a^2-\phi^2(x))$
ReLU	x if $x>0$, else 0	1 if $x>0$, else 0
leaky ReLU	x if $x>0$, else ax	1 if $x>0$, else a

Table 3: Here are popular activation functions used in modern deep learning. For this implementation, Network A will use tanh for both hidden layers while Network B will use leaky ReLU. Both network will use the sigmoid function for the output layer.

On the other hand, back propagation is a little complicated when compared to the forward propagation. The model learns by performing training from the training data. During each epoch (1 cycle of forward and back propagation), we perform forward back propagation to predict a value. However, the model provides an incorrect prediction. The

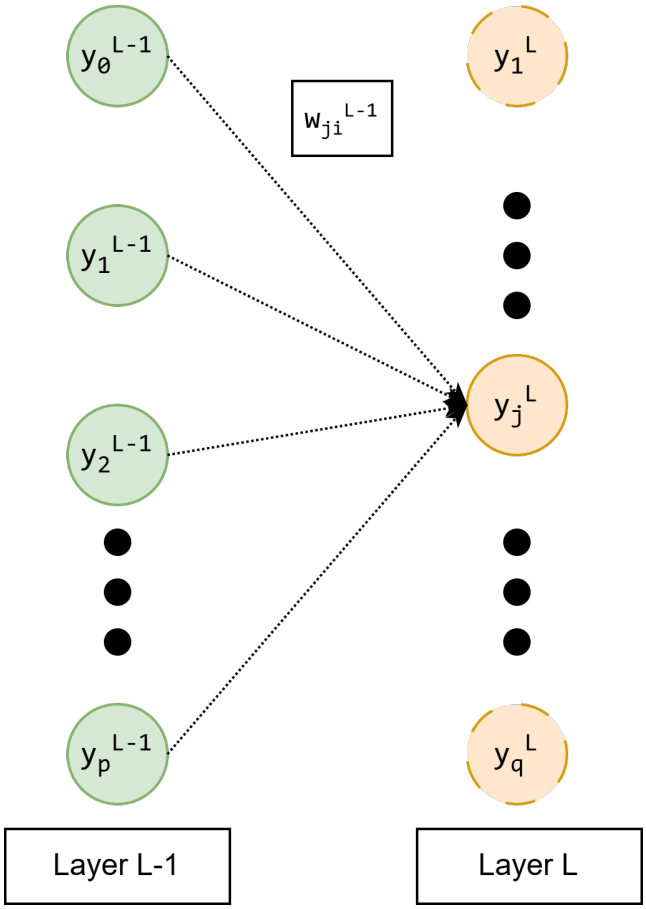

Figure 2: To get the y_j^L, you perform a weighted sum of all y_i^{L-1} from $i = 0$ to $i = p$ and then passed through some activation function ϕ where the index $i = 0$ is the bias term and p is the number of neurons at layer $L - 1$.

magnitude of incorrectness is measured by some loss function. The loss function employed for this implementation is the Mean Squared Error (MSE) which is described in Equation 2. Using the loss, we gradually adjust our weights. The amout of weight adjustment is proportional the the gradient of the loss function with respect to the current weight.

$$L = \frac{1}{2n} \sum_{k=0}^{n} (d_k - y_k)^2 \tag{2}$$

$$W_{n+1}^{L-1} = \begin{cases} W_n^{L-1} + \eta (Y^{L-1})^T \phi'(V^L) \circ e, & \text{outer} \\ W_n^{L-1} + \eta (Y^{L-1})^T \phi'(V^L) \circ (\delta^{L+1} W_n^L), & \text{hidden} \end{cases} \tag{3}$$

The back propagation uses the general delta rule (used for the output layer or for any hidden layer). The vector form of the equation is shown in Equation 3 and visualized at Figure 3. This equation is the result of performing a series of chain rules of different gradients. W_{n+1}^L is the adjusted weight matrix at epoch $n + 1$, W_n^L is the weight matrix before adjustment at epoch n, η is the learning rate which determines how fast the model should learn, $(Y^{L-1})^T$ is the transposed matrix of the neurons, $\phi'(V^L)$ is the derivative of the preactivated neurons, e is the error or difference of the predicted and desired, δ^{L+1} is the local gradient of the previous layer, W_n^L is the weight matrix of the previous layer, and $\circ$ is an element-wise multiplication operation.

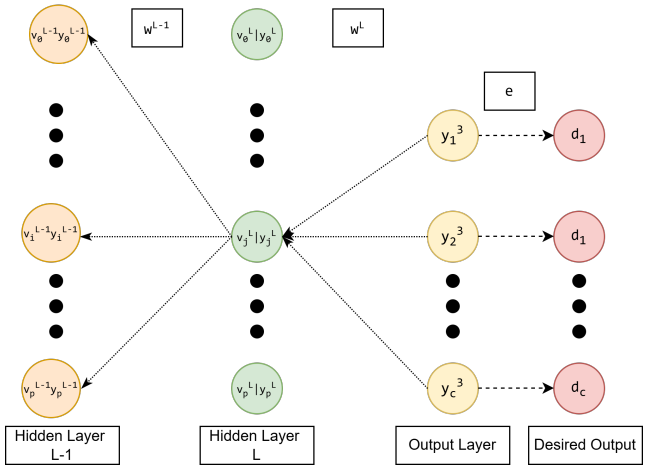

Figure 3: The backpropagation here is used to adjust the new weight matrix at Layer $L-1$. This algorithm starts at the outer layer and propagates the error backwards. The local gradient at the outer layer is defined to be the element wise product of the derivative of the activation and errors. This local gradient term is then stored and used in the next weight adjustment to solve for the new local gradient which is successively used for the next layer.

While at this form, this is already usable. However, it is worth to speed up the training by adding a momentum term described in Equation 4. The momentum term is the weight update from the previous epoch (which is already in storage and need not be computed) multiplied by a constant. This is helpful since if both needed weight updates are in the same direction (two points in the same trend), it speeds up the training, while if both weight updates have opposite signs (a change in trend), it controls the step better.

$$\left(\Delta W_n^L\right)_{\text{no momentum}} = \left(\Delta W_n^L + \alpha \Delta W_{n-1}^L\right)_{\text{with momentum}} \tag{4}$$

Using the algorithms defined above, the MLP model can now be used for training. Lastly, the weight matrices are initialized randomly from a uniform distribution between $[-2.4/F, 2.4/F]$ where F is the number of neurons from the previous layer.

3.3 Hyperparameter Optimization

While training adjusts the model's parameters, or the weights, a default model is generally not good enough. Hence, there is a need to adjust the hyperparameters such as hidden layer size and learning rate to achieve the best model. While it is

impossible to determine the best model due to the sheer size of the hyperparameter space, these hyperparameters can be estimated by searching through these models. The model that gives the best validation accuracy is selected as the best model.

Initially, a model is trained on a learning rate of 0.01 to see a general trend of the learning curve at max epoch of 20. Once that is done, a momentum term is added since it is observed that the model is still capable of learning and at the same time the epochs is increased to 50. Now after observing the learning curve, it is that the model learned faster and the epoch stabilized at around epoch 20. Hence, this epoch is used for the hyperparameter search.

Originally, the hidden layers used (for the two hidden layer MLP) is (256, 128) since they are powers of 2 between the input and output layer sizes, 354 and 8 respectively. The search will happen by getting an interval that contains the original values. The search spaces of the hidden layer 1 size, hidden layer 2 size, and learning rate are $512, 256, 128$, $512, 256, 128$, $0.01, 0.005, 0.001$. This initial search's goal is to determine at what point will have the highest accuracy. Assumption here is that a neighboring point will be the best model. Once the search is done, the a second search is performed using the best combination from the first search, but now on a tighter interval than before.

3.4 Metrics

To evaluate the usability of the trained model, it is common practice to use metrics that can be used to benchmark it against other models (or against the model itself) to see how good it is. Metrics used are the learning curves, Accuracy, Recall, Precision, F1 score, and the Matthew's Correlation Coefficient

3.4.1 Learning Curves

The learning curve is the plot of accuracy and loss against the number of epochs. This is a measure of how well the model is performing throughout the training. The data plotted here are the train loss, train accuracy, validation loss, and validation accuracy. We want the loss to decrease as epochs increases but in a smooth fashion such that there should be minimal oscillations. One thing to be noted is to be careful of the train loss and validation loss curves. Once the validation loss starts to increase, that is a major sign of overfitting and the model starts to not generalize well. However if both curves still decrease, the model still has room to learn from the data. Likewise, for train accuracy and validation accuracy, they should be increasing with epochs. Watch out if validation accuracy trends in the opposite direction and that could be a sign of overfitting.

3.4.2 Accuracy

The accuracy of the model measure how much of the model was predicted correctly over the total number of predictions. While accuracy has its drawbacks for imbalanced dataset, but since the condition has been rectified, accuracy will be the main metric to be of interest.

3.4.3 Precision and Recall

Precision and Recall usually are in a constant trade-off with each other. Precision is a measure of how well a correct positive prediction is made from all positive predictions while Recall is a measure of how well all the correct positive predictions are taken from all the actual positives.

$$P = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \tag{5}$$

$$R = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \tag{6}$$

3.4.4 F1 Score

F1 score is defined as the the harmonic mean of precision and recall. This is common metric since if neither precision or recall is the best metric, one can employ F1 score and have a value in between the two. This metric is also conservative at it leans toward the lower of the two metrics.

$$F1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \tag{7}$$

3.4.5 Matthew's Correlation Coefficient

When not sure what is more costly, a common practice is to use Matthew's Correlation Coefficient or MCC as the formula uses all the valuable metrics (True positive, True negative, False positive, and False Negative). While there are formulas for MCC, shown in Equation 8 is the multiclass formula which uses the confusion matrix. In this equation C_{ij} represents the count of instances of actual class i predicted as class j.

$$\text{MCC} = \frac{\sum_k \sum_l \sum_m C_{kk} C_{lm} - C_{kl} C_{mk}}{\sqrt{\left(\sum_k \left(\sum_j C_{kj}\right) \left(\sum_i C_{ik}\right)\right) \left(\sum_k \left(\sum_j C_{jk}\right) \left(\sum_i C_{ki}\right)\right)}} \tag{8}$$

4. EXPERIMENTAL RESULTS

During a given learning rate and momentum constant, the epoch that provided stable learning curves are during epoch 20 (as shown in Figure 4), hence this is used for hyperparameter optimization. This was selected since it is preferred to choose a relatively low epoch to have more time for search but not small enough that it is unstable during learning.

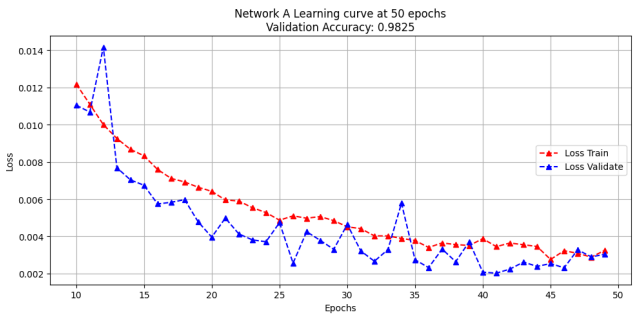

Figure 4: Here, it is observed that epoch 20 is a stable epoch to use for searching hyperparameters.

Once the epoch has been selected, it is now time to do the search. Since there are 3 hyperparameters with 3 values each, the model is trained 27 times with the accuracy extracted. This is called the initial search. Once the best

combination is chosen based on top hyperparamters, then we can perform another tighter search based on the initial search.

For the second search, the search space is reduced to a smaller range containing the one selected from the initial search. The same methodology/approach is also performed on Network B. Shown in Table 8 is the summary of the initial and secondary searches for Network A and Network B. The summary of the search is show in Table 4

	A		B	
	Search 1	Search 2	Search 1	Search 2
HL1	256	256	256	256
HL2 Size	128	140	128	140
η	0.01	0.01	0.01	0.01
acc.	0.96375	0.96625	0.96625	0.9675
Search Time (s)	1961	965	1757	912
Search Time (ave.)	72.6	45.7	65.1	33.8

Table 4: Both are run at 100 epochs, momentum constant at 0.99. Both Network A and Network B are comparable in results. For the run time, Network B ran faster when compared to Network A,

Once the best hyperparameters are determine, it is now time to maximize the model by running it at 100 epochs. Results are shown in Table 5. The results showed that models resulted in the same metrics when both ran at 100 epochs. To validate this, let us check the confusion matrix in Table 6 and Table 7.

	Network A	Network B
Accuracy	0.9825	0.9775
MCC	0.9800	0.9744
Precision @ 1	1.0000	1.0000
Precision @ 2	0.9798	0.9798
Precision @ 3	1.0000	1.0000
Precision @ 4	0.9911	0.9412
Precision @ 5	1.0000	1.0000
Precision @ 6	0.9778	0.9778
Precision @ 7	0.9596	0.9596
Precision @ 8	0.9457	0.9651
Recall @ 1	0.9339	0.9256
Recall @ 2	1.0000	1.0000
Recall @ 3	1.0000	1.0000
Recall @ 4	0.9911	1.0000
Recall @ 5	1.0000	1.0000
Recall @ 6	0.9888	0.9888
Recall @ 7	1.0000	1.0000
Recall @ 8	0.9560	0.9121
F1 @ 1	0.9658	0.9614
F1 @ 2	0.9898	0.9898
F1 @ 3	1.0000	1.0000
F1 @ 4	0.9911	0.9697
F1 @ 5	1.0000	1.0000
F1 @ 6	0.9832	0.9832
F1 @ 7	0.9794	0.9794
F1 @ 8	0.9508	0.9379

Table 5: While the validation accuracy at 20 epochs was slightly different for both Networks, the metrics for the validation data using both models are the same.

Lastly, let us see the learning curves of both Networks. In order, here are the loss curves and accuracy curves for both Network A and Network B. We will be looking at the curves past 30 epochs.

	1	2	3	4	5	6	7	8
1	113	1	0	0	0	0	4	3
2	0	97	0	0	0	0	0	0
3	0	0	86	0	0	0	0	0
4	0	0	0	111	0	0	0	1
5	0	0	0	0	109	0	0	0
6	0	0	0	0	0	88	0	1
7	0	0	0	0	0	0	95	0
8	0	1	0	1	0	2	0	87

Table 6: Confusion matrix for Network A. Row-wise, these are the actual values while column-wise, these are the predictions.

	1	2	3	4	5	6	7	8
1	112	1	0	1	0	1	4	2
2	0	97	0	0	0	0	0	0
3	0	0	86	0	0	0	0	0
4	0	0	0	112	0	0	0	0
5	0	0	0	0	109	0	0	0
6	0	0	0	0	0	88	0	1
7	0	0	0	0	0	0	95	0
8	0	1	0	6	0	1	0	83

Table 7: Confusion matrix for Network B. They are quite similar with Network A for most parts. When compared to Network A, the misclassifications here are mostly 8 being classified as 4.

5. ANALYSIS AND DISCUSSION OF RESULTS

When observing Table 8, it is noticed that learning rates $\eta = 0.1$ usually give good accuracies regardless of hidden layer sizes. Additionally, $\eta = 0.005$ and $\eta = 0.007$ is also acceptable. Other learning rates are either too high that it misses the optimum minimum, or the learning rate is too slow for the given epoch. However, note that only momentum constant of $\alpha = 0.99$ was chosen, hence a higher learning rate could be selected when selecting a lowere momentum constant.

For the hidden layer sizes, there is no clear pattern. A high hidden layer size, low hidden layer size, or mix can give good results or bad. The main driver here is really the learning rate. It may be noticed that our weights initialization depends on the input layer size with respect to the given weight. A larger layer size may initialize your weights into a smaller range (remember the initialization range as (-2.4/F, 2.4/F)). With smaller weights, signals become slow because of low weights. Larger hidden layers increase the number of parameters, potentially slowing convergence and requiring careful initialization.

For the final models of both networks, both agreed to have a layer sizing of (354, 256, 140, 8) and both agreed to have a learning rate of 0.01. This shows that despite what activation function is used, the chances are high that there is a specific range of layer sizes that are optimal.

In terms of run time and search time, the network using leaky ReLU is faster (Network B) since it is not using a function such as tanh. The advantage of using tanh is its ability to give smoother learning curves hence a more stable learning. However, since the gradient (or derivative) of leaky

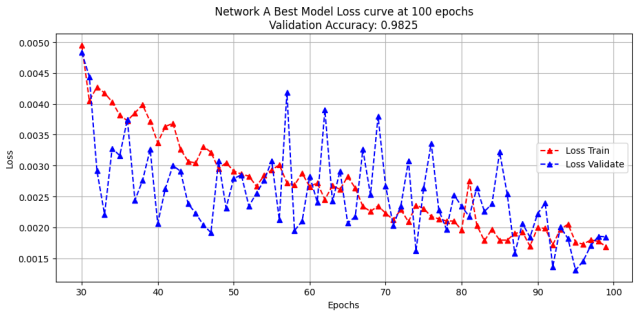

Figure 5: Here is the loss curve for Network A at epochs 30-100. While the validation loss oscillates slightly, the general trend is still with the training loss.

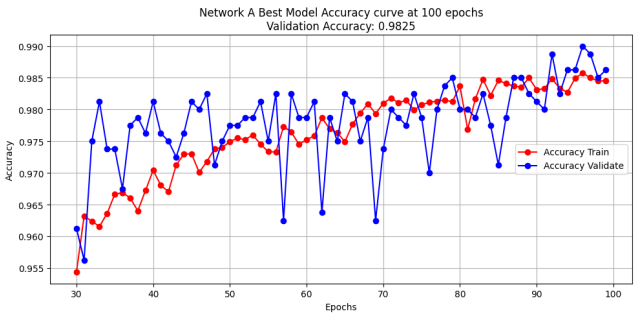

Figure 6: Here is the accuracy curve for Network A at epochs 30-100. While the validation accuracy oscillates slightly, the general trend is still with the training accuracy.

ReLU is constant 1 for positive inputs, it adjusts the weights faster when compared to tanh having small gradients for large inputs. Hence the reason why Network B is much faster!

For the metrics, Network A is the clear winner for the given epoch. It is higher (if not equal) compared to Network B. For the confusion matrix, most of the data that Network A got wrong, Network B also got wrong. The accuracy is quite high, this is due to the fact that the data is trained on balanced data, hence the model is able to extract information equally. MCC is also high, which means that the model is a great predictor. For the precision and recall, there is not minimal trade-off that can be performed since they are almost similar (hence a similar F1 score).

6. CONCLUSION

This paper demonstrates a complete implementation and evaluation of a Multilayer Perceptron (MLP) using forward and backpropagation from purely Numpy. Two versions of the MLP were compared: Network A (using tanh activation functions in hidden layers) and Network B (using leaky ReLU). The primary objective was to assess how different activation functions and hyperparameter (such as hidden layer size and learning rate) configurations affect model performance and training efficiency.

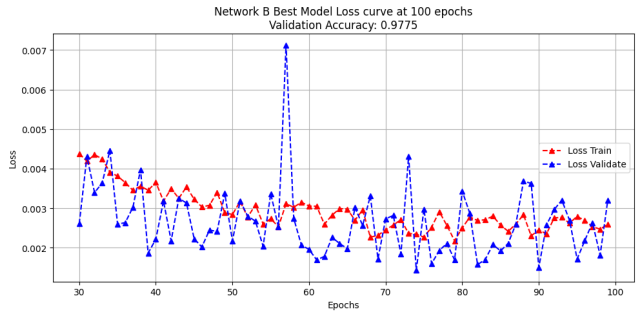

Figure 7: Here is the loss curve for Network B at epochs 30-100. Although the curve spike at 1 instance at around epoch 58, it went back to the curve right after.

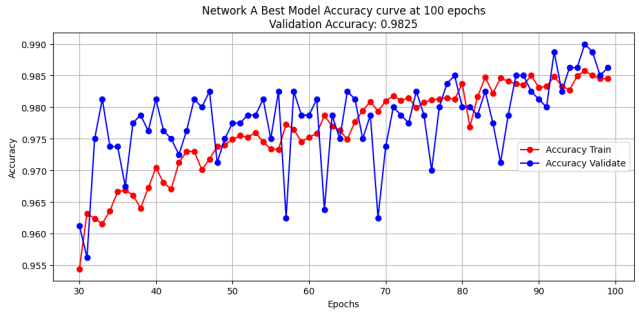

Figure 8: Here is the accuracy curve for Network B at epochs 30-100. While the validation accuracy oscillates slightly, the general trend is still with the training accuracy.

Through systematic data preprocessing, including class balancing with SMOTE and proper train-validation partitioning, the model was made robust against class imbalance. Hyperparameter optimization showed that a learning rate of consistently produced strong results, and momentum further accelerated convergence without destabilizing the learning process. Interestingly, although both networks converged to similar topologies and accuracies, Network B (leaky ReLU) trained significantly faster than Network A (tanh), which is consistent with known properties of leaky ReLU in avoiding vanishing gradients and offering linearity for positive inputs.

From the metrics obtained, both networks achieved high accuracy (above 97%) and high Matthews Correlation Coefficient (above 0.97), validating the effectiveness of the training process and model configuration. Precision, recall, and F1 scores across all classes were consistently strong, further affirming generalization.

In terms of training efficiency, Network B's use of leaky ReLU allowed it to reach comparable accuracy with faster training time, although tanh provided smoother gradients. This highlights a trade-off between computational efficiency and gradient stability, where leaky ReLU is preferred in scenarios requiring faster convergence.

In conclusion, both models performed excellently, but leaky

ReLU emerged as a more computationally efficient choice without sacrificing much predictive power. Future work can extend this study by testing deeper architectures, incorporating regularization techniques, or evaluating performance on real-world noisy datasets. The project overall solidifies the effectiveness of classical MLPs and the impact of careful architectural and hyperparameter decisions on training dynamics.

7. REFERENCES

[1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* New York, NY: Springer, 2009.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] J. Brownlee. (2020) Smote for imbalanced classification with python. Accessed: 2024-04-02. [Online]. Available: https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/

A								B							
Search 1				Search 2				Search 1				Search 2			
HL1	HL2	η	Acc.	HL1	HL2	η	Acc.	HL1	HL2	η	Acc.	HL1	HL2	η	Acc.
512	128	0.01	0.9638	256	140	0.01	0.9663	256	128	0.01	0.9663	256	140	0.01	0.9675
256	128	0.01	0.9638	240	128	0.01	0.9650	512	512	0.01	0.9638	256	128	0.01	0.9663
512	256	0.01	0.9600	280	140	0.01	0.9638	256	512	0.01	0.9638	240	120	0.01	0.9663
128	128	0.01	0.9588	256	128	0.01	0.9638	128	128	0.01	0.9638	256	120	0.01	0.9625
512	512	0.01	0.9475	256	120	0.01	0.9638	128	256	0.01	0.9588	280	120	0.007	0.9575
128	256	0.01	0.9425	240	140	0.01	0.9638	512	128	0.01	0.9575	280	128	0.01	0.9550
512	128	0.005	0.9400	280	128	0.007	0.9563	512	256	0.01	0.9538	240	128	0.01	0.9525
128	128	0.005	0.9400	280	128	0.01	0.9563	256	256	0.01	0.9525	280	140	0.007	0.9513
128	256	0.005	0.9388	256	120	0.007	0.9538	128	512	0.005	0.9463	256	140	0.007	0.9500
256	256	0.01	0.9375	240	128	0.007	0.9538	128	512	0.01	0.9450	240	140	0.01	0.9475
256	128	0.005	0.9363	240	120	0.007	0.9538	128	128	0.005	0.9325	240	140	0.007	0.9438
256	512	0.01	0.9263	256	128	0.007	0.9525	128	256	0.005	0.9275	240	128	0.007	0.9425
256	512	0.005	0.9225	240	140	0.007	0.9500	256	512	0.005	0.9075	280	120	0.01	0.9413
128	512	0.005	0.9225	240	120	0.01	0.9475	256	128	0.005	0.9050	280	128	0.007	0.9400
128	512	0.01	0.9213	280	140	0.007	0.9463	512	128	0.005	0.9000	256	128	0.007	0.9400
256	256	0.005	0.9050	280	120	0.007	0.9450	256	256	0.005	0.8788	256	120	0.007	0.9388
512	256	0.005	0.9000	280	120	0.01	0.9438	512	512	0.005	0.8388	240	120	0.007	0.9325
512	512	0.005	0.8813	256	140	0.007	0.9400	512	256	0.005	0.8175	280	140	0.01	0.9300
128	512	0.001	0.5188	256	128	0.015	0.9338	512	512	0.001	0.1113	256	140	0.015	0.3588
128	128	0.001	0.4675	280	140	0.015	0.9313	512	256	0.001	0.1113	240	140	0.015	0.2663
128	256	0.001	0.4150	280	128	0.015	0.9300	512	128	0.001	0.1113	280	140	0.015	0.2025
512	128	0.001	0.3763	256	120	0.015	0.9275	256	512	0.001	0.1113	280	128	0.015	0.1788
512	256	0.001	0.3238	280	120	0.015	0.9213	256	256	0.001	0.1113	280	120	0.015	0.1513
256	128	0.001	0.2813	256	140	0.015	0.9188	256	128	0.001	0.1113	256	120	0.015	0.1513
256	256	0.001	0.2538	240	128	0.015	0.9125	128	512	0.001	0.1113	240	128	0.015	0.1513
256	512	0.001	0.2225	240	120	0.015	0.8975	128	256	0.001	0.1113	240	120	0.015	0.1513
512	512	0.001	0.1725	240	140	0.015	0.8413	128	128	0.001	0.1113	256	128	0.015	0.1413

Table 8: The combination with the highest accuracy is selected (except for the initial search at Network A since the tie was broken by the more efficient model).